

Curso: DSM

Disciplina: BANCO DE DADOS

Prof^a. Carlos Alberto Vaz

@prof_carlosvaz

INSTAGRAM

Linguagem SQL

SQL é a sigla para *Structured Query Language* (linguagem estruturada para consulta) e contém um conjunto de comandos padronizados para busca e manipulação em SGBDs relacionais.

Transact-SQL, ou simplesmente T-SQL, foi o nome dado a linguagem SQL do Microsoft SQL Server. Além dos comandos SQL padrão ANSI-92 foram incorporadas algumas extensões próprias do produto. Todos os SGBDs possuem suas extensões ao SQL padrão que geram seus *dialetos SQL* próprios, isso é parecido com o regionalismo de nosso idioma.

Por exemplo, no Brasil todos falamos o português que equivaleria ao padrão SQL ANSI-92, porém cada região possui suas expressões próprias que equivaleira ao T-SQL que de certa forma são compreendidas somente naquela região que equivale ao SGBD específico (MS SQL Server, Oracle, Sybase, Informix, DB2, etc.).

Tipos de dados

O SQL Server possui um conjunto de tipos de dados pré-definidos. Esses dados podem ser agrupados em: numéricos, texto, especiais e unicode. A seguir veremos os tipos de dados suportados e suas características

- **Tipos de dados numéricos**

Os tipos de dados numéricos do SQL Server podem ser agrupados em: Numéricos com precisão decimal exata, usados para quantidades e valores, e numéricos com precisão decimal variável, usados para cálculos científicos com valores de ponto flutuante. A tabela a seguir lista todos os tipos de dados numéricos, suas características, bem como indicações de seu uso:

Tipos de dados Numéricos

Tipo	Faixa de valores	Precisão Inteira	Precisão decimal	Bytes	Uso
<i>Decimal</i>	-10^{28-1} a 10^{28-1}	38	37	5 a 17	Para valores com decimais fixas como quantidades e valores
<i>Numeric</i>	-10^{31-1} a 10^{31-1}	38	37	5 a 17	Para valores com decimais fixas como quantidades e valores
<i>Int</i>	-2.147.483.648 a 2.147.483.647	-	-	4	Para valores inteiros com limites dentro da faixa de valores permitida
<i>Smallint</i>	-32.768 a 32.767	-	-	2	Para valores inteiros com limites dentro da faixa de valores permitida
<i>Tinyint</i>	0 a 255	-	-	1	Para valores inteiros com limites dentro da faixa de valores permitida
<i>Money</i>	-2^{53} a 2^{53}	-	4	8	Usado para valores monetários
<i>Smallmoney</i>	-214.748,3648 a 214.748,3647	-	4	4	Usado para valores monetários
<i>Float</i>	$-1.79E + 308$ a $1.79E + 308$	-	7 a 15	4 a 8	Usado para armazenar resultados de cálculos científicos, ou com precisão numérica variável
<i>Real</i>	$-3.40E + 38$ a $3.40E + 38$	-	7 a 15	4	Usado para armazenar resultados de cálculos científicos, ou com precisão numérica variável

Tipos de dados Caracter

Os tipos de dados caráter do SQL Server podem armazenar no máximo 8000 caracteres (letras, números ou símbolos especiais). São os tipos de dados para armazenar nomes, endereços, descrições, observações, etc.

Os tipos de dados caráter podem ser de tamanho fixo (char) ou variável (varchar).

Os tipos de **tamanho fixo** ocupam de espaço no banco de dados exatamente o mesmo número de bytes informado em sua criação; já os tipos de **tamanho variável** economizam o espaço do banco de dados pois só usam os bytes do que está realmente sendo armazenado.

Tipos de dados Character

Por exemplo, se vamos armazenar o nome “José dos Santos Pereira” que contém 23 caracteres em um campo criado como `char(45)` estaremos ocupando 45 bytes de espaço no banco de dados; já se formos armazenar o mesmo nome em um campo criado como `varchar(45)` estaremos ocupando aproximadamente 25 bytes de espaço no banco de dados, ou seja, economizando 20 bytes, o que em um banco de dados com milhões de linhas pode representar uma grande economia.

Tipos de dados Caracter

A tabela a seguir lista todos os tipos de dados caracter de tamanho fixo e variável, suas características, bem como indicações de uso:

Tipo	Tamanho Fixo ou Variável	Uso
<i>Char</i>	Fixo	Para campos do tipo caráter onde estaremos ocupando todo o tamanho reservado para o campo. Por exemplo: Sexo, Sigla de estado, Cor, etc.
<i>Varchar</i>	Variável	Para campos do tipo caráter onde poderemos não estar ocupando todo o tamanho reservado para o campo. Por exemplo: Nome, Endereço, Cidade, Descrição, etc.

SQL

Structured Query Language

Linguagem Estrutura de Consultas

DDL - Data Definition Language
Linguagem de Definição de Dados.

- São os comandos que interagem com os objetos do banco.
- São comandos DDL : CREATE, ALTER e DROP

DML - Data Manipulation Language
Linguagem de Manipulação de Dados.

- São os comandos que interagem com os dados dentro das tabelas.
- São comandos DML : INSERT, DELETE e UPDATE

DQL - Data Query Language
Linguagem de Consulta de dados.

- São os comandos de consulta.
- São comandos DQL : SELECT (é o comando de consulta)

DTL - Data Transaction Language
Linguagem de Transação de Dados.

- São os comandos para controle de transação.
- São comandos DTL : BEGIN TRANSACTION, COMMIT E ROLLBACK

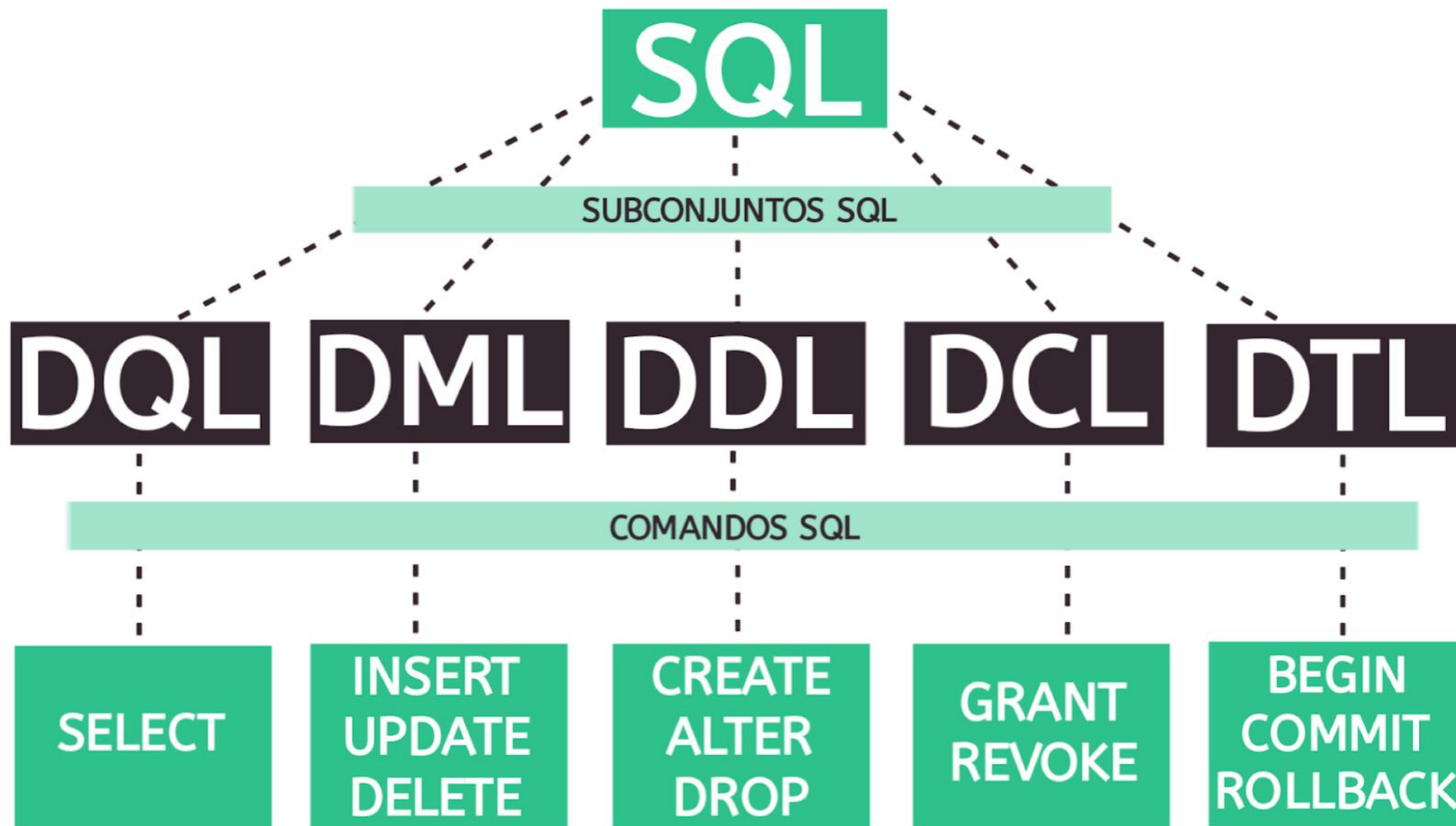
DCL - Data Control Language
Linguagem de Controle de Dados.

- São os comandos para controlar a parte de segurança do banco de dados.
- São comandos DCL : GRANT, REVOKE E DENY.

SQL

Structured Query Language

Linguagem Estrutura de Consultas





Tipos de Dados do SQL SERVER



Todas vez que você define uma coluna para capturar um atributo de objeto, é necessário definir um tipo de dados.



O tipo de dados é um atributo de cada coluna que especifica o tipo de dados que a coluna armazenará.



Por exemplo, talvez você queira que a coluna Nome armazene valores de string (alfanuméricos), que a coluna Preço armazene dados monetários, etc...

Validando os dados

NOT NULL: é equivalente ao valor
REQUERIDO: SIM;

DEFAULT: permite declarar um valor
de coluna se nenhum for especificado
quando a linha (registro) é inserido;

PRIMARY KEY: declara a coluna como
chave primária;

IDENTITY: utilizado para numerar
automaticamente, geralmente
utilizado nas PK. É necessário informar
qual o valor inicial e o valor do
incremento, por exemplo (1,1), inicia
no número 1, incrementando de 1 em
1.

UNIQUE: Faz com que o valor do
registro não possa ser repetido;

CHECK: Limita o valor do registro a um
intervalo de valores ou a uma
determinada condição;

FOREIGN KEY: Limita os valores de
uma coluna àqueles que podem estar
localizados em outras tabelas. Por
exemplo, se criar uma coluna
codDepto na tabela de Funcionarios,
você quer que o bando de dados
garanta que esse departamento exista
na tabela de Departamentos;

Comandos para definição de dados (DDL)

Os comandos para definição de dados, **DDL (*Data Definition Language*)** são os comandos para criação, alteração e exclusão de objetos em bancos de dados SQL Server.

Usamos comandos DDL para criar bancos de dados, tabelas, índices, views, stored procedures, tipos de dados definidos pelo usuário (UDTs) e qualquer objeto que faz parte de uma aplicação no servidor de bancos de dados.

Comandos para definição de dados (DDL)

Por exemplo, para criar uma tabela de Funcionarios no banco de dados Empresa, que contenha o código do funcionário, seu nome completo e cargo na empresa usamos o comando DDL CREATE:

```
CREATE TABLE Funcionarios  
(FunCodigo INT,  
FunNome VARCHAR(40),  
FunCargo VARCHAR(30));
```

Comando CREATE

Cria um objeto no SQL Server. Objetos podem ser: Bancos de dados, tabelas, views, stored procedures, views, índices, tipos de dados definidos pelo usuário (UDTs), regras (ROLES), valores padrão (DEFAULTS).

Comando ALTER

Altera a definição de um objeto. Usado por exemplo para criar novos campos em tabelas, aumentar o tamanho de um banco de dados, alterar a chave de um índice, ou alterar o código de uma view.

Utilizado para eliminar uma tabela do banco de dados. Elimina a estrutura e registros.

Ex.

DROP TABLE CLIENTES;

--Elimina a tabela clientes

Comando de consulta (DQL)

Os comandos para consulta de dados, DQL (*Data Query Language*), são os comandos usados para seleção e manutenção de dados em tabelas ou views. São os comandos mais utilizados na construção de aplicações usando SGBDs

O comando DQL possuem uma sintaxe SQL padrão (ANSI), porém cada produto implementou extensões próprias a esses comandos aumentando sua funcionalidade. A vantagem de se usar a sintaxe SQL padrão é que uma mesma aplicação pode dessa forma acessar diferentes SGBDs. Durante esse curso veremos em detalhes todo o comando DQL e demais comandos, principalmente o comando SELECT que possui uma alta funcionalidade para seleção de dados.

Comando de consulta (DQL)

Por exemplo, para se obter uma lista de nomes e cargos dos Funcionarios do banco de dados Empresa cujo cargo seja Diretor usamos o comando DQL **SELECT** da seguinte forma:

```
SELECT FunNome, FunCargo FROM Funcionarios  
WHERE FunCargo = 'Diretor'
```

Comando SELECT

SELECT é o principal comando da linguagem SQL de qualquer SGBD, pois é através dele que selecionamos os dados armazenados em nossos bancos de dados, o que, em última análise é a finalidade de sistemas de informação baseados em bancos de dados.

Sua funcionalidade é bastante abrangente e iremos ver mais detalhes sobre o mesmo nos tópicos seguintes. A sintaxe básica do comando SELECT é:

Comando SELECT

SELECT[lista de campos]

FROM [lista de tabelas]

WHERE [critérios de seleção]

Comando SELECT

Cláusula	Conteúdo	Descrição
SELECT	[Lista de campos]	É a parte do comando onde informamos o nome dos campos das tabelas que queremos visualizar. Podemos também criar campos virtuais para visualização, que serão resultado de funções ou cálculos sobre campos reais.
FROM	[lista de tabelas]	Nomes das tabelas que contém os dados que desejamos visualizar. Nessa parte do comando vão as cláusulas de junção entre tabelas.
WHERE	[critérios de seleção]	Contém os critérios de seleção de registros das tabelas. Na sintaxe ANSI nessa parte do comando entram os operadores de junção de tabelas.

Selecionando colunas

Na primeira parte do comando SELECT declaramos o nome dos campos que desejamos acessar separados por vírgula. Os campos serão mostrados na ordem em que forem declarados.

Para selecionar todos os campos de uma tabela pode ser usado o símbolo * (asterisco).

Selecionando colunas

Exemplo 1: Seleciona todas os campos da tabela Titulos no banco de dados Biblioteca:

```
SELECT * FROM Titulos
```

Exemplo 2: Seleciona os campos Data_reserva e isbn da tabela Reservas no banco de dados Biblioteca:

```
SELECT Data_reserva, isbn FROM Reservas
```

Selecionando linhas com a cláusula WHERE

Podemos selecionar os registros ou linhas que desejamos obter através da cláusula WHERE. Para fazer a seleção usamos os operadores de comparação, lógicos, de faixas ou lista de valores, para valores nulos e comparação para textos.

Usando operadores

O SQL Server possui um conjunto de operadores para comparação, seleção de texto, lógicos, etc. O uso de operadores está sempre associado a um critério de seleção que é a função principal da cláusula WHERE. Serão descritos a seguir os operadores usados em T-SQL.

Operadores de Comparação

Os operadores de comparação são aqueles que checam igualdade, diferença e intervalos de valores.

Operador	Descrição
=	igual
>	maior
>=	Maior ou igual
<	Menor
<=	Menor ou igual
<>	Diferente

Comando SELECT

Exemplo 3: Seleciona todos os campos e os registros onde o campo titulo_no seja igual a 9, da tabela Titulos do banco de dados Biblioteca:

```
SELECT * FROM Titulos WHERE titulo_no = 9
```

Exemplo 4: Seleciona todos os campos e os registros onde o campo titulo_no seja maior que 47, da tabela Titulos do banco de dados Biblioteca:

```
SELECT * FROM Titulos WHERE titulo_no > 47
```

Comparações com texto

Para efetuarmos comparações com campos texto, os mesmos operadores de comparação descritos anteriormente podem ser usados. Além disso, no Transact-SQL existe um operador especial para buscas em texto, o LIKE. Quando usamos o operador LIKE podemos fazer a busca em textos usando um padrão. Esse padrão é formado pelos seguintes símbolos:

Comando SELECT

Símbolo	Descrição
%	Representa qualquer conjunto de caracteres na posição.
–	Representa qualquer caráter na posição.
[-]	Representa a checagem da existência de um caráter específico ou faixa de caracteres na posição.
[^–]	Representa a checagem da não existência de um caráter específico ou faixa de caracteres na posição.

Exemplo 5: Seleciona somente o campo Autor e os registros onde o campo autor seja igual = “Spinoza”, da tabela Titulos do banco de dados Biblioteca:

```
SELECT Autor FROM Titulos WHERE autor = ‘Spinoza’
```

Exemplo 6: Seleciona somente os campos nome e cidade, e os registros onde o campo nome comece com “Da”, da tabela Membros do banco de dados Biblioteca :

```
SELECT nome,cidade FROM Membros WHERE nome LIKE ‘Da%’
```


Exemplo 7: Selecciona somente os campos nome e cidade, e os registros onde o campo nome não tenha como segundo carácter as letras L,M ou N , da tabela Membros do banco de dados Biblioteca:

```
SELECT nome,cidade FROM Membros WHERE nome LIKE ‘_[^L-N]%
```

Operadores Lógicos

Como em qualquer outra linguagem, o T-SQL possui um conjunto de operadores lógicos. Os operadores lógicos servem para avaliar várias condições em uma mesma operação de busca. O operador lógico de negação (NOT) avalia a negação de uma expressão.

Símbolo	Descrição
AND	Seleciona os registros que satisfazem a TODAS as condições
OR	Seleciona os registros que satisfazem a ALGUMA condição
NOT	Seleciona os registros que NÃO satisfazem a condição.

Comando SELECT

A precedência na análise dos operadores é feita da seguinte forma:

- Primeiro é feito a análise de expressões dentro de parênteses “()” do mais interno para o mais externo;
- Os operadores lógicos são analisados na ordem : NOT, AND e OR;
- Os operadores são analisados da esquerda para a direita.

Exemplo 8: Seleciona somente o campo Autor e os registros onde o campo autor seja igual = 'Spinoza' ou = 'Motojirou', da tabela Titulos do banco de dados Biblioteca :

```
SELECT Autor FROM Titulos  
WHERE autor = 'Spinoza' OR autor = 'Motojirou'
```

Comando SELECT

Exemplo 9: Seleciona somente os campos membro_no, nome e cidade, e os registros onde o campo nome não comece com “Ca” e a cidade seja = “Sacramento”, ou o membro_no esteja entre 1 e 200, da tabela Membros do banco de dados Biblioteca:

```
SELECT membro_no,nome,cidade FROM Membros  
WHERE (nome NOT LIKE 'Ca%' AND cidade = 'Sacramento') OR  
(membro_no BETWEEN 1 AND 200)
```

Operador IN

O operador IN permite fazer a seleção de um valor dentro de uma lista de valores. Seu uso equivale a várias expressões lógicas OR.

Exemplo 10: Selecciona somente o campo Autor e os registros onde o campo autor seja igual = “Spinoza” ou “Motojirou”, da tabela Titulos do banco de dados Biblioteca:

```
SELECT Autor FROM Titulos WHERE  
autor IN(‘Spinoza’, ‘Motojirou’)
```

Operador BETWEEN

Para buscas dentro de uma faixa de valores o operador BETWEEN pode ser usado. O operador BETWEEN equivale a uma expressão com os operadores $\geq$ (maior ou igual) e $\leq$ (menor ou igual).

Exemplo 11: Seleciona todos os campos e os registros onde o campo Data_devolucao esteja entre os dias 20 e 22 de março de 2017 na tabela Emprestimos do banco de dados Biblioteca:

```
SELECT * FROM Emprestimos
```

```
WHERE data_devolucao BETWEEN '20170320' AND '20170322'
```

Ordenação de resultados

O resultado do comando **SELECT** pode ser ordenado (classificado) através da cláusula **ORDER BY**. A ordenação pode ser ascendente (**ASC**), que é o padrão, ou descendente (**DESC**); podemos usar os nomes dos campos ou sua posição na lista de campos para determinar os campos para ordenação. Os campos usados para ordenação devem fazer parte da lista de campos.

Comando SELECT

Exemplo 13: Seleciona todos os campos, e todos os registros da tabela Titulos do banco de dados Biblioteca ordenados pelo campo autor:

SELECT * FROM Titulos ORDER BY autor

Exemplo 14: Seleciona todos os campos, e os registros onde o membro_no = 3094 da tabela Emprestimos do banco de dados Biblioteca ordenados por data_saida decendente:

**SELECT * FROM Emprestimos WHERE membro_no = 3094
ORDER BY data_saida DESC**

Comando SELECT

Exemplo 15: Selecciona os campos isbn,membro_no e data_saida, e todos os registros da tabela Emprestimos do banco de dados Biblioteca ordenados por isbn ascendente, e data_saida descendente:

```
SELECT isbn,membro_no,data_saida FROM Emprestimos  
ORDER BY 1 ASC, 3 DESC
```

Eliminação de duplicidades

A duplicidade de linhas no resultado de comandos SELECT pode ser eliminada com o uso da cláusula DISTINCT.

Exemplo 16: Selecciona o campo cidade, e todos os registros da tabela Membros do banco de dados Biblioteca eliminando duplicidades no campo cidade:

```
SELECT DISTINCT cidade FROM Membros  
ORDER BY cidade
```

Funções de agregação

O SQL possui um conjunto de funções para agregação de dados. Essas funções são:

Função	Descrição
COUNT	Conta a ocorrência do campo. COUNT(*) retorna o número de registros da tabela.
SUM	Totaliza os valores para o campo. Usado com campos numéricos.
MAX	Devolve o maior valor para o campo.
MIN	Devolve o menor valor para o campo.
AVG	Média aritmética simples dos valores do campo.

Funções de agregação

Exemplos:

Considerar uma tabela de Produtos com os seguintes atributos:

Codprod int, nome varchar(40), qtde int, preco Numeric(10,2).

- Quantos produtos temos no cadastro ?

```
SELECT COUNT(*) AS 'QTDE' FROM PRODUTOS;
```

A cláusula **AS** permite dar um apelido para a coluna de resultados.

Funções de agregação

- Qual a quantidade total de produtos no cadastro?

```
SELECT SUM(qtde) AS 'QTDE TOTAL' FROM PRODUTOS;
```

- Qual o preço mais caro?

```
SELECT MAX(preco) AS 'MAIS CARO' FROM PRODUTOS;
```

- Qual o preço mais barato?

```
SELECT MIN(preco) AS 'MAIS BARATO' FROM PRODUTOS;
```

Funções de agregação

- Qual a média de preços dos produtos?

```
SELECT AVG(preco) AS 'MÉDIA' FROM PRODUTOS;
```

Curso: DSM

Disciplina: BANCO DE DADOS

Prof^a. Carlos Alberto Vaz

@prof_carlosvaz

INSTAGRAM

Muito Obrigado!!!